

Дополнение

Утв $P \neq NP \Rightarrow$ для задачи TSP (коммивояжёр) порог аппроксимации равен 1.

$\square$ $\exists$ порог ~~аппр~~ $\epsilon < 1$ и $\exists$ М-ПОНТ, которая ϵ -аппроксимирует TSP.

Хотим получить противоречие таким образом: решим задачу ДНАМРАТН за полиномиальное время, что невозможно при условии $P \neq NP$, т.к. ДНАМРАТН $\in$ NPC.

Для этого исходный граф G преобразуем следующим образом: дополним его рёбрами до полного графа. Всем рёбрам, которые были в G , зададим вес 1, а остальным L . Такое L мы выберем потом достаточно большим.

Заметим, что если в G был гамильтонов путь из s в t , то коммивояжёр, чтобы пройти из s в t оптимальнее (то есть минимизировать вес пути)

надо воспользоваться гамильтоновым путём, т.к. $L > 1$. То есть в этом случае

$OPT = |V| - 1$, где $|V|$ - число вершин в G . Но М-алгоритм ϵ -аппроксимации, то есть длина $\forall$ другого найденного пути $= L$ удовлетворяет неравенству:

$$\frac{|L - OPT|}{\max(1, OPT)} \leq \epsilon \quad \text{В силу того, что } L \geq OPT \text{ получим:}$$

$$\frac{L - OPT}{L} \leq \epsilon$$

Т.к. L - не гамильтонов путь, то в нём содержится хотя бы одно ребро веса L , т.е. $L \geq L + |V| - 2$. Заметим, что при увеличении

L тогда $\frac{L-OPT}{L}$ увеличивается, поэтому подставим минимальное возможное:

$$\frac{L+|V|-2 - |V|+1}{L+|V|-1} \leq \varepsilon$$

$$\frac{L-1}{|V|+L-1} \leq \varepsilon$$

$$L-1 \leq \varepsilon|V| + \varepsilon(L-1)$$

$$(1-\varepsilon)(L-1) \leq \varepsilon|V|$$

$$L \leq \frac{\varepsilon|V|}{1-\varepsilon} + 1$$

Тогда $L \geq \left\lceil \frac{\varepsilon|V|}{1-\varepsilon} \right\rceil + 2$ и всех больших, при наличии гамильтонова пути из $s \in V$ найдёт гарантированно его. То есть мы можем при $L = \left\lceil \frac{\varepsilon|V|}{1-\varepsilon} \right\rceil + 2$ запустить M на G' и получим, что длина пути равна $|V|-1$, если в G был гамильтонов путь из $s \in V$, и что-то большее иначе. То есть мы научились решать ИКАВРАТИ за полиномиальное время, пользуясь M , что невозможно при условии $P \neq NP$ ■

Сейчас будет обоснование, возможно, неочевидного перехода в неравенстве при доказательстве амплификации BPP. А именно, будет чёткое обоснование того, что $(p(1-p))^{\frac{n}{2}} \cdot \sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} p^t (1-p)^{-t} \leq (p(1-p))^{\frac{n}{2}} \cdot 2^n$.

Для этого докажем, что: $\sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} \left(\frac{p}{1-p}\right)^t \leq 2^n$

Т.к. $p < \frac{1}{2}$, то $1-p > \frac{1}{2} \Rightarrow \frac{p}{1-p} < 1$. Т.к. $t \geq 0$, то $\left(\frac{p}{1-p}\right)^t \leq 1 \quad \forall t \in 0 \dots \frac{n}{2}$

Тогда $\sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} \left(\frac{p}{1-p}\right)^t \leq \sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} \leq \sum_{t=0}^n C_n^t$ (добавили $\frac{n}{2}$ слагаемых) =

$= 2^n$. Лемма доказана.

Теперь пару слов про амплификации других классов. Класс RP не поддаётся

амплификации, то есть для него фиксированная таблица. В классе RP можно поменять вероятность ошибки $\frac{1}{2}$ на $\forall p \in (0; 1)$. Амплифицируется

он следующим образом: мы также запускаем МТ n^m раз, но про класс RP

известно, что если нам вывелось 1, то слово точно в языке. То есть, если

все n^m запусков выдали 0, то мы выводим 0, и тут возможна ошибка,

если хотя бы одна 1 вывелась, то выводим 1, и слово точно в языке.

Посчитаем вероятность ошибки.] у нашего класса вероятность ошибки

$p \in (0; 1)$, т.е. при $x \in L, P(V(x; r) = 0) \leq p$. Тогда ошибка после n^m запусков -

это когда $x \in L$, и на всех запросах МТ ошиблась, т.е. $\leq p^{n^m} \rightarrow 0 \Rightarrow$

$\Rightarrow$ амплифицировали RP.

Теорема $RP \cap L \in NL$

□ 1) Сначала докажем, что $RP \cap L \in NL$. Воспользуемся сертификатным определением NL. В качестве сертификата передадим сам путь из s в t , то есть

последовательность вершин. Верификатор будет брать пару соседних вершин и проверять, что между ними действительно есть ребро.

Нам не надо проверять, что вершины на в этом пути не повторяются, потому что если есть сложный путь из s в t , то есть и простой (достаточно из текущего убрать все циклы).

2) Теперь докажем, что $DPATH$ - NL -трудный. Выберем произвольный $A \in NL$.

Хотим $A \leq_L DPATH$. Т.к. $A \in NL$, то $\exists$ НМТ M_A , работающая на логарифмической памяти, распознающая A . Посчитаем количество

конфигураций M_A . $\exists$ у M_A 2 ленты: входная с x только для чтения, и рабочая, на которой есть $O(\log|x|)$ ячеек для работы. Тогда количество конфигураций на входной ленте: $|Q||x|$, а на рабочей $|Q|^{O(\log|x|)} \cdot O(\log|x|) \cdot |Q|$. Итого $|Q|^2 \cdot |x| \cdot O(\log|x|) \cdot |Q|^{O(\log|x|)} = O(\text{poly}(|x|))$ конфигураций.

То есть размер графа конфигураций имеет размер $O(\text{poly}(|x|))$. При этом на задании одной конфигурации можно тратить $O(\log|x|)$ памяти, т.к. конфигурацию рабочей ленты можно ^{хранить} ~~записать~~ целиком, а для входной ленты достаточно сохранять только положение считывающего устройства и символ состояния: $|Q|\log|x|$ памяти. Итого, $\mathcal{F}$, вычисленная на логарифмической памяти, перебирает все пары вершин u и v графа конфигураций, по таблице переходов проверяет, можно ли за один шаг из u попасть в v , и если да, то выводит ребро $u \rightarrow v$. На каждом

сразу мы храним 2 вершины и их айдишники, скажем так. Т.к. в графе
 полиномиальное число вершин, то на хранение айдишников требуется тоже
 логарифмическая память. Итак, S на логарифмической памяти вывел граф
 G . Осталось вершины s и t . Если наложить условия на M на фиксированные
 начальную и принимающую конфигурации, то надо просто вывести соответствующие
 им вершины. $x \in A \Leftrightarrow S(x) = (G, s, t) \in \text{DPATH} - \text{ID}$ в силу эквивалентности
 задач $\Rightarrow \forall A \in \text{NL} \text{ и } A \in \text{DPATH} \Rightarrow \text{DPATH} - \text{NC-трудный} \Rightarrow \text{DPATH} \in \text{NCC} \blacksquare$